Implementação 4 - MetroREC

1. Objetivo

O objetivo dessa atividade é desenvolver um sistema de sincronização do embarque de passageiros em um metrô usando sincronização (por mutex) e threads. O meu programa controla a espera dos passageiros, a disponibilidade de lugares nos vagões e a conclusão dos embarques, respeitando as regras estabelecidas no enunciado.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
README + Makefile	OK	Readme está disponivel no github e o uso do makefile é evidenciado rodando: make clean e make
Relatório	OK	é este documento, também está no github
Implementação	OK	teste rodando: make clean make ./sync_car

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar:

```
./sync_car
```

4. Arquitetura

Arquivos e responsabilidades:

1. **sync_car.h → Declaração da struct station e protótipos das funções.**
2. **sync_car.c → Implementação das funções de sincronização.**
3. **sync_car-runner.c → Arquivo de testes fornecido pelo professor para verificar o funcionamento da implementação.**

Decisões técnicas:

1. **Utilizar contadores e variáveis de condição** → Escolhi essa abordagem por ser mais simples de implementar e entender → Dessa forma, consigo controlar os passageiros esperando, os lugares disponíveis e os embarques pendentes sem precisar criar estruturas mais complexas.
2. **Utilizar três contadores para controlar o embarque** → Escolhi utilizar passageiros_esperando, lugares_livres e aguardando_embarcar para acompanhar cada etapa do embarque → Dessa forma, consigo verificar quantos passageiros estão esperando, quantos lugares ainda estão disponíveis e quantos passageiros precisam confirmar o embarque antes que o vagão possa partir.
3. **Controlar a saída do vagão utilizando uma única condição no while** → Escolhi verificar se ainda existem passageiros esperando com lugares disponíveis ou embarques pendentes → Assim, o vagão só encerra o embarque quando não houver mais passageiros que possam ocupar os lugares e todos os embarques autorizados tiverem sido confirmados.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Contadores, mutex e variáveis de condição	Primeiro estudei a implementação e depois fui implementando uma função por vez com a premissa de três contadores para controlar os passageiros e os lugares disponíveis, um mutex para proteger as informações e duas variáveis de condição para coordenar o embarque.	

5.2 Diário de Tentativas

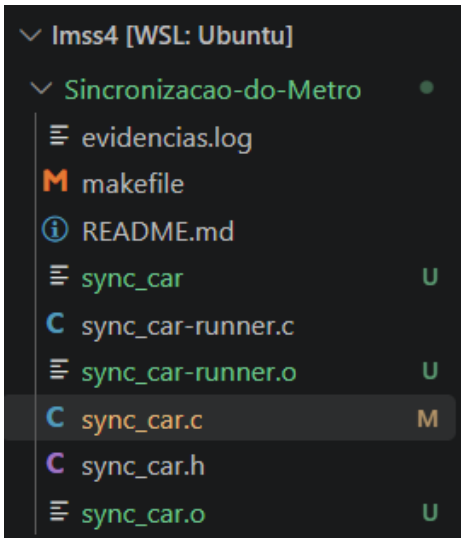
#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Compilar o código após finalizar as funções	Falhou	Baguncei a organização das funções, acabei nomeando elas errado e não implementei station_on_board	04/11	Evidencias.log Linhas: 94 a 108
2	S1	Compilar o código com as alterações feitas	Parcial	O código compila mas está gerando executáveis “.o” e na implementação ele só pede pra gerar 1 executavel sync_car	04/11	Print 1
3	S1	Rodar o teste passado pelo professor no sync_car-runner	falhou	A sincronização do vagão com os passageiros estava errada, eu chamei pthread_cond_wait	04/11	Print 2

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
				sem bloquear o mutex antes.		
4	S1	Rodar várias entradas do sync_car-runner	Parcial	estava tendo uma condição de corrida causada pela alteração de variáveis compartilhadas fora da proteção do mutex na função station_load_car, fazendo com que o vagão não concluísse o embarque no tempo esperado.	04/11	evidencias.log Linhas: 325 a 333
5	S1	Testes extras	Passou		04/11	evidencias.log Linhas: 644 a 679

6. Evidências

Print 1:

Antes:




```
sync_car: sync_car.o sync_car-runner.o
    gcc sync_car.o sync_car-runner.o -o sync_car -pthread

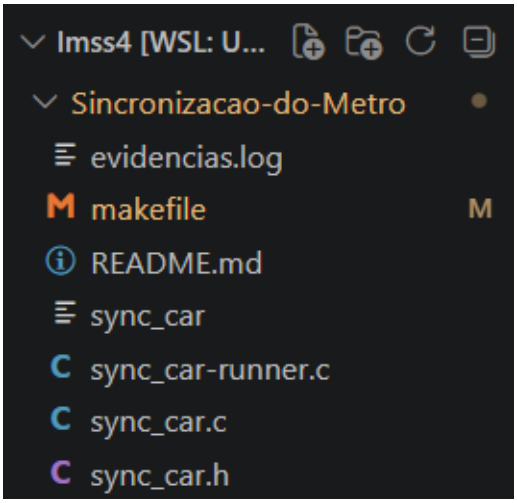
sync_car.o: sync_car.c sync_car.h
    gcc -Wall -Wextra -pthread -c sync_car.c

sync_car-runner.o: sync_car-runner.c sync_car.h
    gcc -Wall -Wextra -pthread -c sync_car-runner.c

clean:
    rm -f *.o sync_car

.PHONY: clean
```

Depois:




```
sync_car: sync_car.c sync_car-runner.c sync_car.h
    gcc -Wall -Wextra -pthread sync_car.c sync_car-runner.c -o sync_car

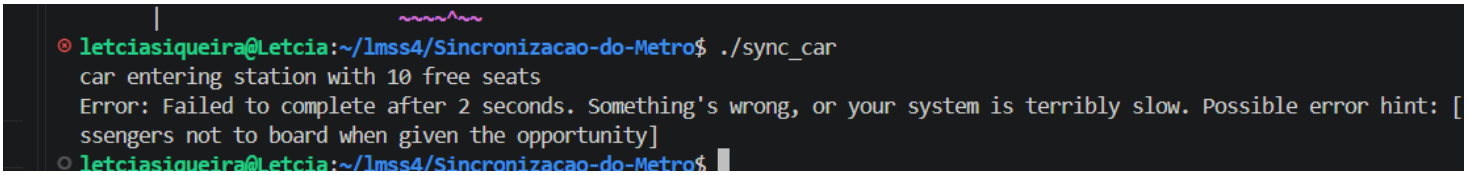
clean:
    rm -f sync_car *.o

.PHONY: clean
```

Explicação da resolução: Meu makefile estava gerando os executáveis **.o** , então alterei o makefile para gerar apenas o executável sync_car. (Não sei se isso é exatamente um erro, mas como na atividade ele pedia para gerar apenas o sync_car como executável resolvi levar ao pé da letra e realmente só gerar ele).

Print 2:

Antes (saída e código):




```
void station_wait_for_car(struct station *station) {

    station->passageiros_esperando++;

    while (station->lugares_livres == 0) {
        pthread_cond_wait(&station->embarque_liberado, &station->mutex);
    }

    station->passageiros_esperando--;
    station->lugares_livres--;
    station->aguardando_embarcar++;

}
```

Depois (saída e código):

```
letciasiqueira@Letcia:~/lmss4/Sincronizacao-do-Metro$ ./sync_car
car entering station with 1 free seats
car departed station with 1 new passenger(s) (expected 1)
car entering station with 34 free seats
car departed station with 34 new passenger(s) (expected 34)
car entering station with 15 free seats
car departed station with 15 new passenger(s) (expected 15)
car entering station with 24 free seats
car departed station with 24 new passenger(s) (expected 24)
car entering station with 26 free seats
car departed station with 26 new passenger(s) (expected 26)
Looks good!
```

```c
void station_wait_for_car(struct station *station) {

    pthread_mutex_lock(&station->mutex);

    station->passageiros_esperando++;

    while (station->lugares_livres == 0) {
        pthread_cond_wait(&station->embarque_liberado, &station->mutex);
    }

    station->passageiros_esperando--;
    station->lugares_livres--;
    station->aguardando_embarcar++;

    pthread_mutex_unlock(&station->mutex);

}
```

Explicação da resolução: Como não tinha bloqueado o mutex quando se chama essa função o comportamento é indefinido e aí isso acarretou em comportamentos incorretos. Então apenas adicionei o bloqueio e desbloqueio do mutex no início e no final da função respectivamente.

7. Uso de IA

Onde usei IA (2–3 linhas): Utilizei IA para entender a atividade (como usar as funções de mutex que foi pedido, estudar sobre sincronização e etc.), Utilizei IA para gerar testes para testar meu código, algumas vezes para entender porque um erro estava acontecendo e para me ajudar a fazer o readme.

Prompts principais (liste, sem história):

- Como funciona o mutex e as funções que foram pedidas para serem usadas?
- Me explique melhor como a sincronização se aplica nessa atividade
- Me ajude a construir o readme do meu repositório

O que validei manualmente e como: Fui testando as coisas que ela ia me explicando e tentando implementar do meu jeito. Sempre que algo dava errado eu tentava entender e achar o erro para depois mandar para ela. Sempre dentro do que foi pedido na atividade.

8. Reflexão Final

Reflexão:

Consegui entender melhor como funciona a sincronização entre threads e a importância do mutex para proteger informações compartilhadas. Minha principal dificuldade foi entender quando precisava bloquear e desbloquear o mutex, o que acabou causando erros nas primeiras execuções. Também percebi que fazer o programa compilar e passar em um teste não significa que ele está totalmente correto, já que alguns problemas só apareceram quando executei o runner várias vezes. Por isso, realizei testes adicionais com diferentes situações de embarque, incluindo casos-limite e concorrência entre passageiros. Se eu fosse fazer

essa atividade denovo eu teria mais atenção ao uso do mutex desde o início e faria testes menores durante a implementação de cada função, em vez de testar tudo apenas depois de finalizar o código.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	SIM
Rodei os testes e evidencias.log foi gerado	SIM
Tenho prints obrigatórios	SIM
Testei pelo menos um caso limite e um caso inválido	SIM
Preenchi a seção de uso de IA	SIM
Revisei o relatório e removi frases genéricas / vazias	SIM

10. Se eu tivesse mais 2 horas

Eu faria testes mais específicos de concorrência, tentando identificar situações em que várias threads acessam a estação praticamente ao mesmo tempo. Uma melhoria seria acrescentar verificações dos retornos de pthread_mutex_init e pthread_cond_init, evitando que o programa continue executando caso algum recurso não seja inicializado corretamente. Acho que revisaria os avisos da compilação e repetiria os testes de estresse, comparando os resultados para verificar se alguma falha aparece depois de várias execuções. Assim, conseguiria aumentar a robustez da implementação sem precisar deixar o código muito mais complexo. No geral acho que meu código está bom.